

Lab 3: Basic Linux network programming

The goal of this assignment is to learn how to write simple programs, using Python, for the reception and generation of network traffic under Linux. You will learn what are and how to use sockets, with a focus on sending and receiving data over a Wi-Fi network with UDP and TCP.

Prerequisite

- An Access Point (AP) for accessing Internet and connecting two devices to the same Wi-Fi network.
- A laptop running Linux, with a Wi-Fi card up and running on it: if you have no Linux, download a live image and run it from a USB stick, as seen before lab 1 [1].
- A second laptop (or PC) with a Wi-Fi card to connect to the same AP as the one of the first laptop.

Software Tools

During this laboratory activity you will mainly work with Python code. Therefore, the main software you will need is:

- **python**: more specifically, we will focus on Python 3.
- **pip3**: this is the Python 3 package manager that can be used to install additional libraries and packages on Python; you will not work directly with **pip**, unless you find out that you are missing some packages that you may need.
- To write Python code, you can use any Integrated Development Environment (IDE) of your choice. The suggestion, as you are likely running a live Linux USB, is to use the **Sublime Text** editor that provides syntax highlighting for Python, while being at the same time a lightweight programming IDE. You can install it following the instructions in [2] or by running the following commands in the Ubuntu terminal (**each command after '\$' must be written over a single line**):

```
$ wget -qO - https://download.sublimetext.com/sublimehq-pub.gpg |  
  gpg --dearmor | sudo tee /etc/apt/trusted.gpg.d/sublimehq-  
  archive.gpg > /dev/null  
$ echo 'deb https://download.sublimetext.com/ apt/stable/' | sudo  
  tee /etc/apt/sources.list.d/sublime-text.list  
$ sudo apt update  
$ sudo apt install apt-transport-https  
$ sudo apt install sublime-text
```

Any Python program can be launched with the command `python3 ./<name_of_python_script>.py`, where `<name_of_python_script>` is the name of your Python program.

- A more powerful alternative to Sublime Text, is **Visual Studio Code** [3]. Visual Studio Code is a free and highly customizable source code editor developed by Microsoft that supports multiple programming languages (including Python) and operating systems, being available for Windows, macOS and Linux. It can be installed under Linux by downloading the **.deb** package from <https://code.visualstudio.com>, and then by running, from the directory when you saved the package (**each command after '\$' must be written over a single line**):

```
$ sudo apt update  
$ sudo apt install ./<deb_file_name>.deb
```

- **tc qdisc** [4]: this is a Linux tool that can be used to manipulate traffic control settings; combined with the **netem** functionality, it can be used to simulate packet loss/delay in a network for experimentation purposes.
- **wireshark** [5]: Wireshark is a free and open-source packet analyzer.

Individual groups with motivated special needs

Only if you are alone in the group due to a motivated special need, and you do not have two Linux devices to connect to the same Access Point, you can also use two Linux Virtual Machines (VMs) ran on your laptop/desktop PC. **If this is the case, motivate it in your report.**

To do so, you can use the VirtualBox virtualization software, and prepare two Ubuntu VMs (any version between 22 LTS, 24 LTS or 26 LTS should be fine). One machine will act as *device 1*, and the other as *device 2*. A good tutorial that you can follow to prepare the two Virtual Machines is the following: <https://ubuntu.com/tutorials/how-to-run-ubuntu-desktop-on-a-virtual-machine-using-virtualbox#1-overview>

The two Virtual Machines will not have a Wi-Fi interface, but you should be able to use the virtualized Ethernet interface (which information and addresses can be obtained with `ip addr`) as an alternative for your experiments, as if it was a Wi-Fi interface. The virtualized Ethernet interface will have an IP address both for the *device 1* VM and for the *device 2* VM, that you can use instead of the Wi-Fi IP address that you would have when working with real devices connected to an Access Point.

Installing a programming environment

First of all, you need to select and install a Python programming environment. Choose Sublime Text, Visual Studio Code or any other editor you may be familiar with¹. Install the editor. **Report which editor did you choose, with a small screenshot.**

Basic communication with UDP sockets

We will first start with some basic experiments with UDP sockets.

To set up the environment for the experiments, take two laptops running Linux and connect them to the same Access Point. From now on, we will call them *device 1* and *device 2*. To avoid connecting to a very crowded Wi-Fi network, the suggestion is to use, if possible, a **mobile hotspot** or a **home Wi-Fi** rather than the public Wi-Fi available at university.

Use then `ip addr` to get the **Wi-Fi** IP address of both laptops. **Which is the assigned IP address to both devices? Report the output of `ip addr` considering only the lines with the IP addresses of both devices.**

We will now create two Python programs using **UDP sockets**:

- One client program to be executed on *device 1*, that should send to *device 2* 30 UDP packets, one every 200 ms, via **Wi-Fi**, containing a string (without spaces) of your choice followed by a comma, a sequence number and the \$ character to terminate the string (as an example, the payload for packet 15 will be: `UDP_experiment,15$`). We will use, as an example, a destination port equal to **47999**.
- One server program to be executed on *device 2*, that should receive any UDP packet received via the **Wi-Fi interface**, on the port you selected before. For each received packet it should print the string, the sequence number, and the number of packets that have been received so far. As buffer size, you can choose for example **512** bytes, as it is large enough to accommodate each packet you are sending.

Remember that, to be able to pass a string to `sendto()` you need first to `encode()` it into a bytes object. Furthermore, to parse a message that has been received with `recvfrom()` as a string, you need first to decode it. As an example:

```
data, addr = sock.recvfrom(1024)
data_str = data.decode()
print(data_str) # data_str contains a string with the content of the message
```

¹However, the suggestion would be to choose either the simpler Sublime Text, or the more powerful Visual Studio Code.

Your **client program** on *device 1* should look like this. Remember to replace the destination IP address (<YOUR_SERVER_IP>) with the one of the server program running on *device 2*, and the MESSAGE_STRING with a message or your choice (<YOUR_STRING>).

```
import socket
import time

UDP_IP = "<YOUR_SERVER_IP>"
UDP_PORT = 47999
MESSAGE_STRING = <YOUR_STRING>

print("UDP server IP: %s" % UDP_IP)
print("UDP destination port: %s" % UDP_PORT)

count = 0 # Counter to count the number of packets

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) # IPv4 + UDP

while count < 30:
    count = count + 1
    message = MESSAGE_STRING + "," + str(count) + "$"
    print("sending message: %s" % message)
    sock.sendto(message.encode(),(UDP_IP, UDP_PORT))

    time.sleep(0.2) # Wait 200 ms before sending the next packet
```

Your **server program** on *device 2* should look like this. Remember to replace the bind IP address (<YOUR_BIND_IP>, i.e., the IP address of the interface where to receive data from) with the one of the Wi-Fi interface of *device 2*.

```
import socket

UDP_IP = "<YOUR_BIND_IP>"
UDP_PORT = 47999

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) # IPv4 + UDP
sock.bind((UDP_IP, UDP_PORT))

count = 0 # To count the number of packets received so far
while True:
    data, addr = sock.recvfrom(512) # data contains the packet payload
    count = count + 1
    # The next line separates the received string using ','
    fields = data.decode().split(",")
    print("received message: %s, %s, %d" % (fields[0],fields[1],count))
```

Launch now the receiving program on *device 2* first, and then the transmitting program on *device 1*. Is everything working as expected? Which output are you getting? Report it and briefly comment it. Explain also, with a brief bullet point list, the main steps that are followed by the client and server programs to create the socket and prepare/send/receive the packets.

Open now **Wireshark** on *device 2* and make it listen to the Wi-Fi interface (remember that you need to launch it with `sudo wireshark` – here **no monitor mode is needed**). Launch again both the receiving and transmitting programs. **Can you see the UDP packets? Which filter can you use to easily find the UDP packets transmitted by device 1? Look at one of the packets sent by device 1. Can you identify the source and destination IP and MAC addresses? Do they correspond to what you see with *ip addr* on both devices? Furthermore, can you see the payload of the packet? Does it correspond to the content you programmed on device 1? Attach a small screenshot.**

Look now at the source and destination ports. **Which values are you seeing? Why?**

Try now editing the transmitting client program on *device 1*, and change the destination port (UDP_PORT), keeping the same port as before on the receiving server program on *device 2*. Repeat the experiment while Wireshark is capturing on the destination *device 2*.

Can the Python program on *device 2* receive the packets? Can you see them, instead, on Wireshark? Briefly comment the output that you get both from Wireshark and from Python.

After doing this experiment, restore the right port on the client on *device 1*.

Basic communication with TCP sockets

Starting from what you did in the previous section, write now two programs similar to the ones you wrote for the previous part of the laboratory activity, but using **TCP** instead of UDP:

- One client program to be executed on *device 1*, that should send to *device 2* 30 TCP packets, one every 200 ms, via **Wi-Fi**, containing a string (without spaces) of your choice followed by a comma, a sequence number and the \$ character to terminate the string (as an example for packet 15: `TCP_experiment,15$`). Use a destination port of your choice. **Report the port that you selected.**
- One server program to be executed on *device 2*, that should receive any TCP packet received via the **Wi-Fi interface**, on the port you selected before. For each received packet it should print the string, the sequence number, and the number of packets that have been received so far. It is not required for the server to reply back to the client. Select also a suitable buffer size for reception.

Important note: we have seen in the TCP server example that you can receive packets via TCP with a code similar to the following, after accepting a connection from a client:

```
[...]
while True:
    data = conn.recv(1024) # 1024 bytes buffer size
    if not data: break # Exit the loop when the client disconnects
    print("received message: %s" % data.decode()) # Print received data

    # If needed, you can reply here to the client
    [...]
conn.close()
```

However, notice how in this case the TCP client should send its packets every 200 ms, without waiting for any confirmation from the server. In this specific case, if the network is unreliable and loses packets, or if it has a latency greater than 200 ms (as it may happen on some crowded/congested Wi-Fi networks), it may happen that, due to retransmissions and to the mandatory hop through the AP, two packets are received very close to each other at the server side, even if they were sent spaced by 200 ms by the client, and they are buffered together at the receiving side.

When this happens, the `recv()` function may return two or more payloads one “attached” to each other. For example, if we send two packets containing, respectively, `TCP_experiment,3$` and `TCP_experiment,4$`, and the second packet for some reason takes more than 200 ms to be successfully transmitted, the `recv()` function may return a single string inside `data`, i.e., `TCP_experiment,3$TCP_experiment,4$`. This string, however, does not correspond to a single packet, but to two different packets that were buffered together. Therefore, this case must be taken into account **when using TCP**.

Thanks to the \$ character that we add at the end of each payload, we can detect when two (or more) packets are received at once and properly consider the related payloads as coming from two different packets, as follows:

```
[...]
while True:
    data = conn.recv(1024)
    if not data:
        break

    # Split payloads if received together
    # This line of code also removes the '$' at the end
    messages = data.decode().split("$")

    # Loop over all received payloads
    # If a single packet is received at a time, the for loop
    # will iterate over a single payload
    for message in messages[:-1]:
        count = count + 1
        print("Received message: %s, %d" % (message, count))
conn.close()
```

What we are doing here is not to immediately print the received data, but first to split the received string (after decoding it) at the \$ character. `messages` will contain then a list of all the payloads that have been received. In case a single packet has been received at once (as expected in most cases), the code will behave just like in the previous case, counting a new received packet and printing the content of the payload as a string. In case multiple payloads are received at once, the `for` loop will iterate over the whole list of received `messages` and count each one as a separate packet, thanks to the \$ character we added as separator.

Write now the code for the TCP client and server following the instructions reported above. There is no need to report the code you have written! However, describe briefly, with a bullet point list, which are the main differences with respect to UDP. Launch then the receiving program on *device 2* first, and then the transmitting program on *device 1*. Is everything working as expected? Which output are you getting? Attach a small screenshot.

Open now Wireshark on *device 2* and make it listen to the Wi-Fi interface. Launch again both the receiving and transmitting programs. **Can you see the TCP packets exchanged by your applications? Which filter can you use to easily identify them? Can you locate the initial handshake, the data exchange and the final connection termination? Attach one or more screenshots of Wireshark showing these three phases (keep the screenshot(s) readable, but with a limited image size, to stay in the 5 pages).**

Try now editing the transmitting program on *device 1*, and change the destination port, keeping the same port as before on the receiving program on *device 2*. Repeat the experiment while Wireshark is capturing on the destination *device 2*.

By looking at the application output and at Wireshark, what happens? Which difference you notice with respect to UDP when the client uses a wrong destination port?

After doing this experiment, restore the right port on the client on *device 1*.

Communication in unreliable networks with packet loss

We will now perform an experiment to show the behavior of UDP and TCP sockets when an unreliable network is involved. To place ourselves in a conditions that clearly shows the differences between the two protocols, we are going to simulate a heavy packet loss at an Operating System level, since it is unlikely that just two devices connected to the same AP would experience a high-enough packet loss. The same packet loss conditions could be experienced in very crowded environments, or in mobile networks with a huge number of devices involved.

To simulate packet loss over a given interface, it is possible to rely on the `tc qdisc` Linux tool,

with the `netem` functionality, used to emulate packet loss and delay for testing purposes.

Let us add a 25% packet loss, by running on *device 1*:

```
$ sudo tc qdisc add dev INTERFACE_NAME root netem loss 25%
```

Where `INTERFACE_NAME` should be replaced with the name of the Wi-Fi interface of *device 1*.

Run then again the applications you wrote in the previous two sessions, first using UDP and then TCP. **Which is the difference between UDP and TCP?** Looking at the sequence number and at the total number of received packets printed by the server, **can you estimate the packet loss? Which is the packet loss for UDP and TCP?**

At the end of this experiment, restore the interface normal operations with:

```
$ sudo tc qdisc del dev INTERFACE_NAME root netem
```

Groups and reporting

You are expected to work on groups of up to four students. Each group is required to prepare a **single** report describing results of this lab. This report must not exceed 5 pages.

Deadlines:

- Delivery of the report: June 3rd 2026

Please upload all your reports to the portal – do not send me them by email!

Your group reports should be uploaded in PDF format and with the name shown at the beginning of the labs, i.e., lab_03_group_XX.pdf, where XX is your group number over two digits (01 for group 1, 02 for group 2, ..., 41 for group 41).

A single report should be submitted by each group.

Wrong naming of the report will result in a 1 point penalization.

References

- [1] Mobile and Sensor Networks. Lab 1: Basic Wi-Fi scanning and Bluetooth tools.
- [2] https://www.sublimetext.com/docs/linux_repositories.html
- [3] <https://code.visualstudio.com/>
- [4] <https://netbeez.net/blog/how-to-use-the-linux-traffic-control/>
- [5] <https://www.wireshark.org/docs/man-pages/tshark.html>